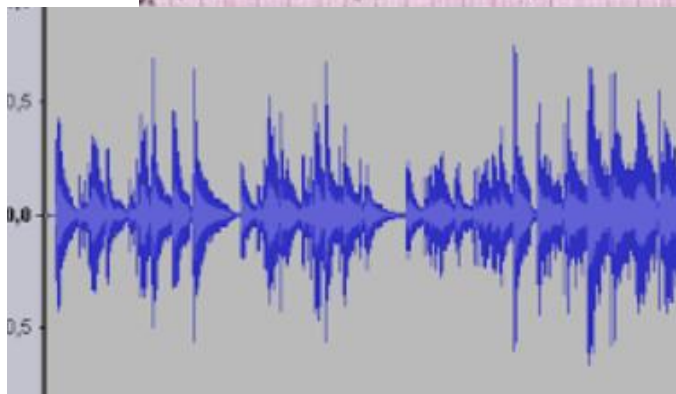


Wykład 6

Sieci Rekurencyjne (RNN)

Sieci LSTM

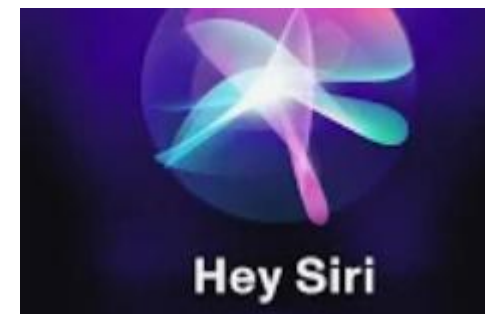
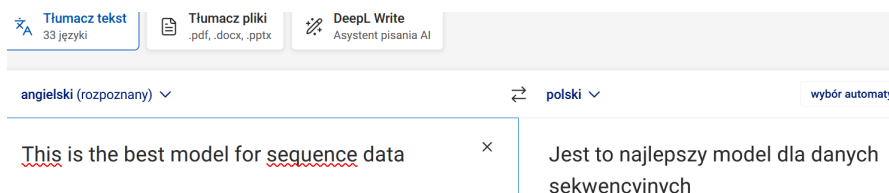
Modelowanie danych sekwencyjnych

[illegible]

Google Voice Search



Idea sieci pamięci długotrwałej została wprowadzona przez Hochreitera.



Deep Learning in Neural Networks: An Overview

Technical Report IDSIA-03-14 / arXiv:1404.7828 v4 [cs.NE] (88 pages, 888 references)

Jürgen Schmidhuber

The Swiss AI Lab IDSIA

Istituto Dalle Molle di Studi sull'Intelligenza Artificiale

University of Lugano & SUPSI

Galleria 2, 6928 Manno-Lugano

Switzerland

8 October 2014

Trochę historii

- W roku 1982 Hopfield publikuje przełomową pracę.
- Opisuje w niej rekurencyjną sieć neuronową działającą jako pamięć skojarzeniowa. Ta praca przekonuje setki naukowców , matematyków, fizyków i inżynierów, aby ponownie zająć się badaniami nad sieciami neuronowymi.

Trochę historii

LSTM jako udoskonalenie sieci RNN.

Idea sieci pamięci długotrwałej została wprowadzona przez Hochreitera w 1991 roku w jego pracy magisterskiej, ale pierwsza publikacja na temat sieci LSTM pochodzi z 1997 roku i jest dziełem Hochreitera i Schmidhubera (Hochreiter, S. i Schmidhuber, J., 1997).

Model sieci LSTM został wprowadzony jako ulepszenie sieci RNN, w których problem znikających lub eksplodujących gradientów opisany w 1994 roku pojawił się przy długich sekwencjach danych, które mogą mieć setki lub nawet tysiące kroków (Bengio, Y., Simard, P. i Frasconi, P., 1994).

Jürgen Schmidhuber



Schmidhuber speaking at the AI for GOOD
Global Summit in 2017

Born 17 January 1963^[1]
Munich,^[1] West Germany
Alma mater [Technical University of Munich](#)
Known for [Long short-term memory](#), [Gödel machine](#), artificial curiosity, [meta-learning](#)

Sepp Hochreiter



Hochreiter in 2012

Born 14 February 1967 (age 58)
[Mühldorf, West Germany](#)
Nationality [German](#)
Alma mater [Technische Universität München](#)
Scientific career

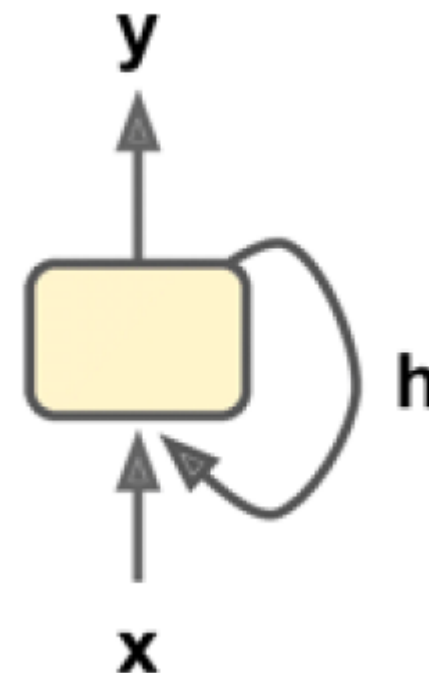
Rekurencyjne sieci neuronowe-zastosowania

- analiza szeregów czasowych, np. cen akcji w celu predykcji kupna/sprzedaży
- przewidywanie trajektorii samochodów autonomicznych w celu unikania wypadków
- przetwarzanie języka naturalnego, np.:
 - automatyczne tłumaczenie,
 - konwersja tekstu na mowę (speech-to-text)
 - analiza sentymentu

RNN są nazywane rekurencyjnymi/powtarzającymi się, ponieważ wykonują to samo zadanie dla każdego elementu sekwencji, a dane wyjściowe zależą od poprzednich obliczeń.

Najprostszy neuron RNN

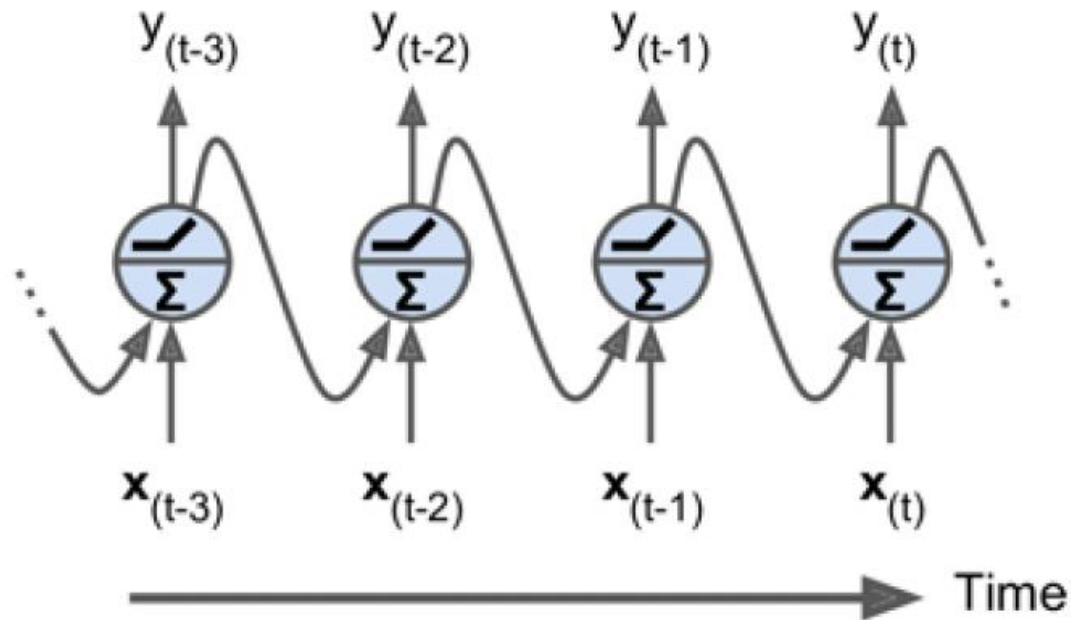
W każdym kroku czasowym t (zwanym również ramką) neuron rekurencyjny odbiera wejścia $x(t)$ oraz własne wyjście z poprzedniego kroku czasowego $y(t-1)$.



Źródło: Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, O'Reilly Media, 2017

Rekurencyjny neuron rozwinięty w czasie

- Możemy reprezentować sieć na osi czasu



Rozwinięcie neuronu RNN w czasie

3 typy rekurencyjnych sieci neuronowych

- RNN
- LSTM
- GRU

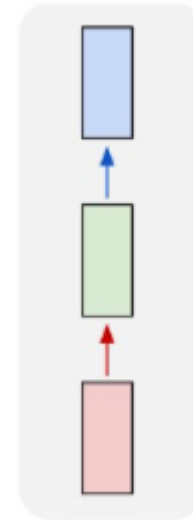
Architektura RNN

jeden do jednego:

Tutaj jest tylko jedna para.

Architektura jeden do jednego jest używana w tradycyjnych sieciach neuronowych.

one to one



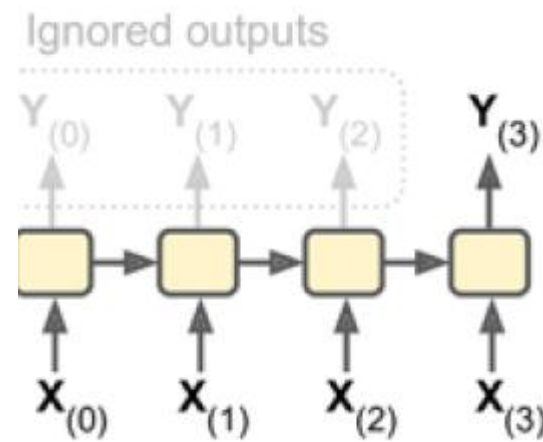
Architektury RNN

Many To One:

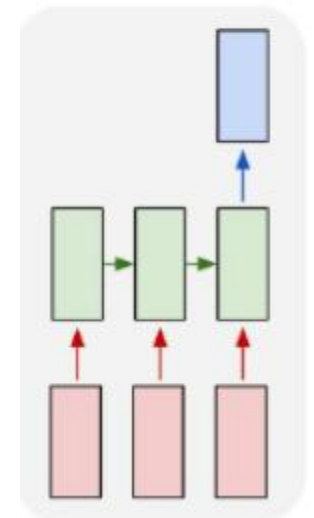
W tym scenariuszu pojedynczy wynik jest generowany przez połączenie wielu danych wejściowych z różnych kroków czasowych.

Analiza sentymentów i identyfikacja emocji wykorzystują takie sieci, w których etykieta klasy jest określana przez sekwencję słów.

Np. recenzja filmu: wynik ocena



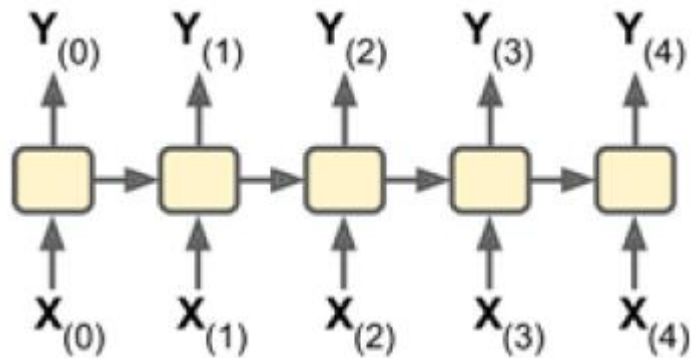
many to one



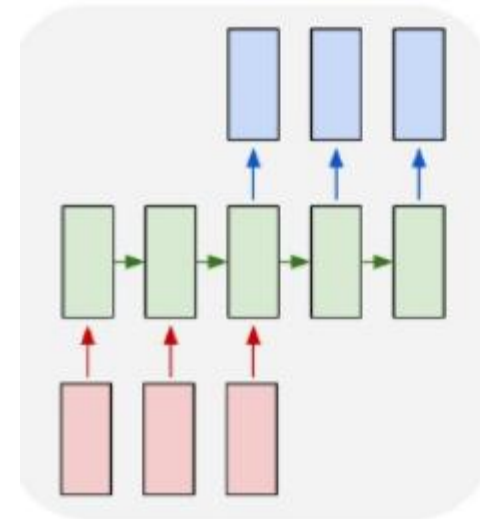
Architektury RNN

Wiele do wielu: W przypadku wielu do wielu istnieje wiele opcji.

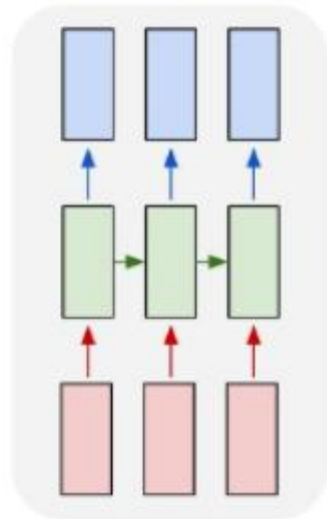
Zastosowanie: Systemy tłumaczenia maszynowego, takie jak systemy tłumaczenia z angielskiego na francuski lub odwrotnie, wykorzystują sieci wiele do wielu.



many to many



many to many



 Tłumacz tekst
33 języki

 Tłumacz pliki
.pdf, .docx, .pptx

 DeepL Write
Asystent pisania AI

angielski (rozpoznany) ▾

↔ polski ▾

wybór automat

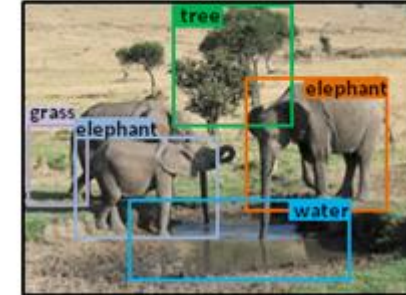
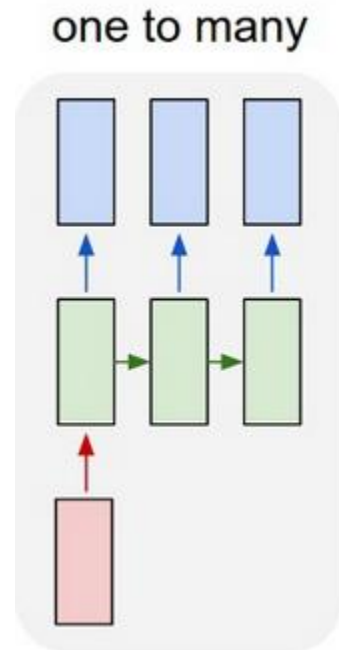
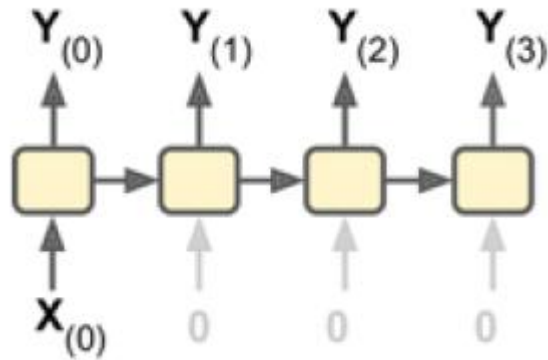
This is the best model for sequence data ×

Jest to najlepszy model dla danych sekwencyjnych

Architektury RNN

Jeden do wielu: Pojedyncze wejście w sieci jeden do wielu może skutkować wieloma wyjściami.

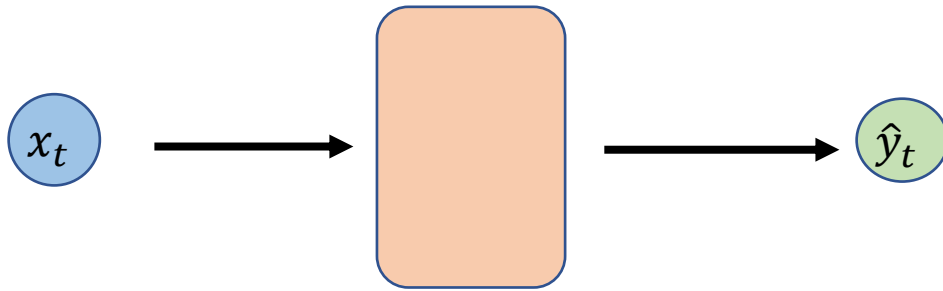
Sieci jeden do wielu są wykorzystywane na przykład w produkcji muzyki lub tworzeniu opisów zdjęć.



BT: three elephants standing in the grass with **water**.

Base: a group of elephants standing on the grass.

Sieć jednokierunkowa-powtórzenie



Wejście/input sieci

Wynik/output sieci

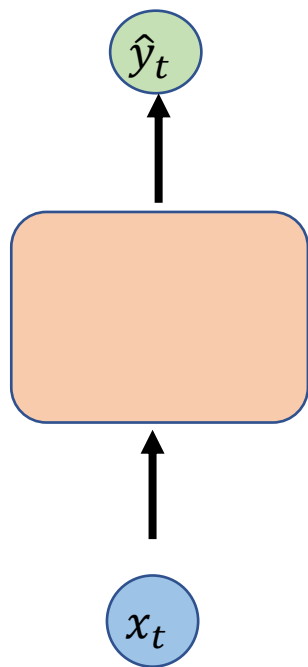
$$x_t \in \mathbb{R}^m$$

$$\hat{y}_t \in \mathbb{R}^n$$

$$\hat{y}_t = f(x_t)$$

Architektura FFN

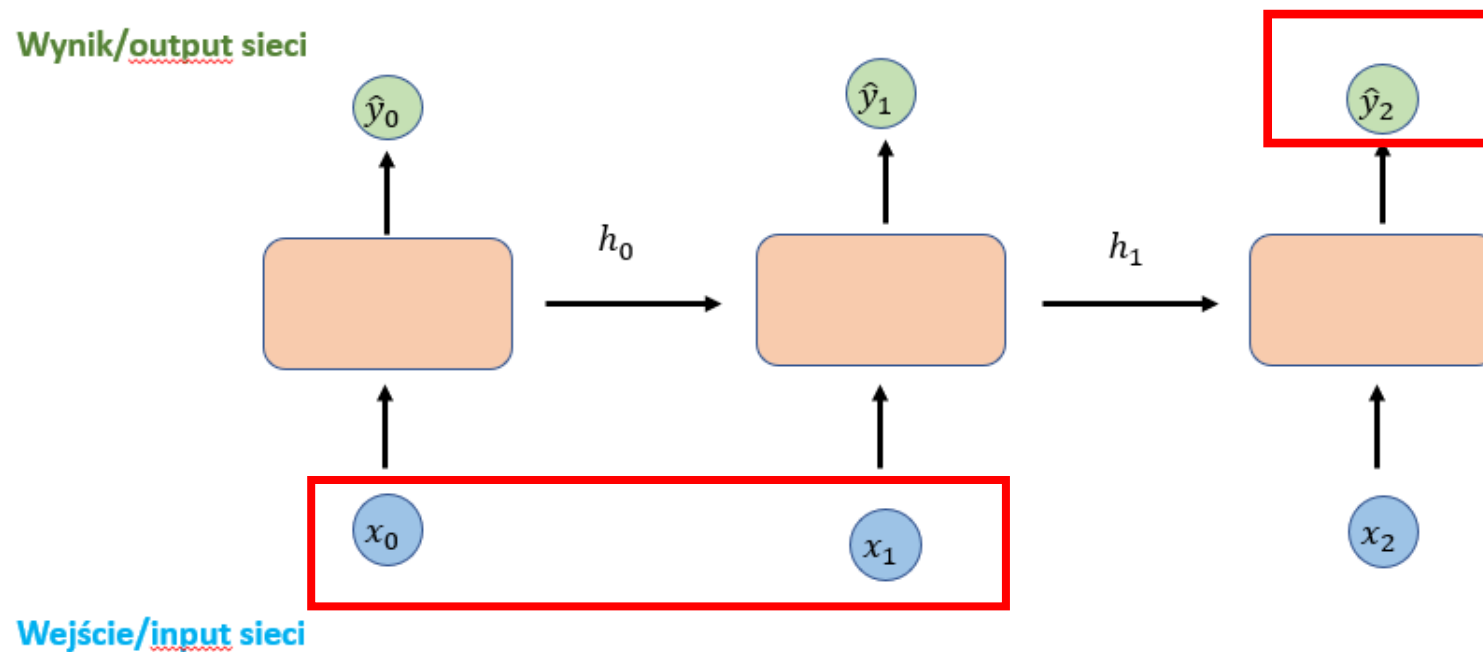
Wynik/output sieci



Wejście/input sieci

Architektura RNN

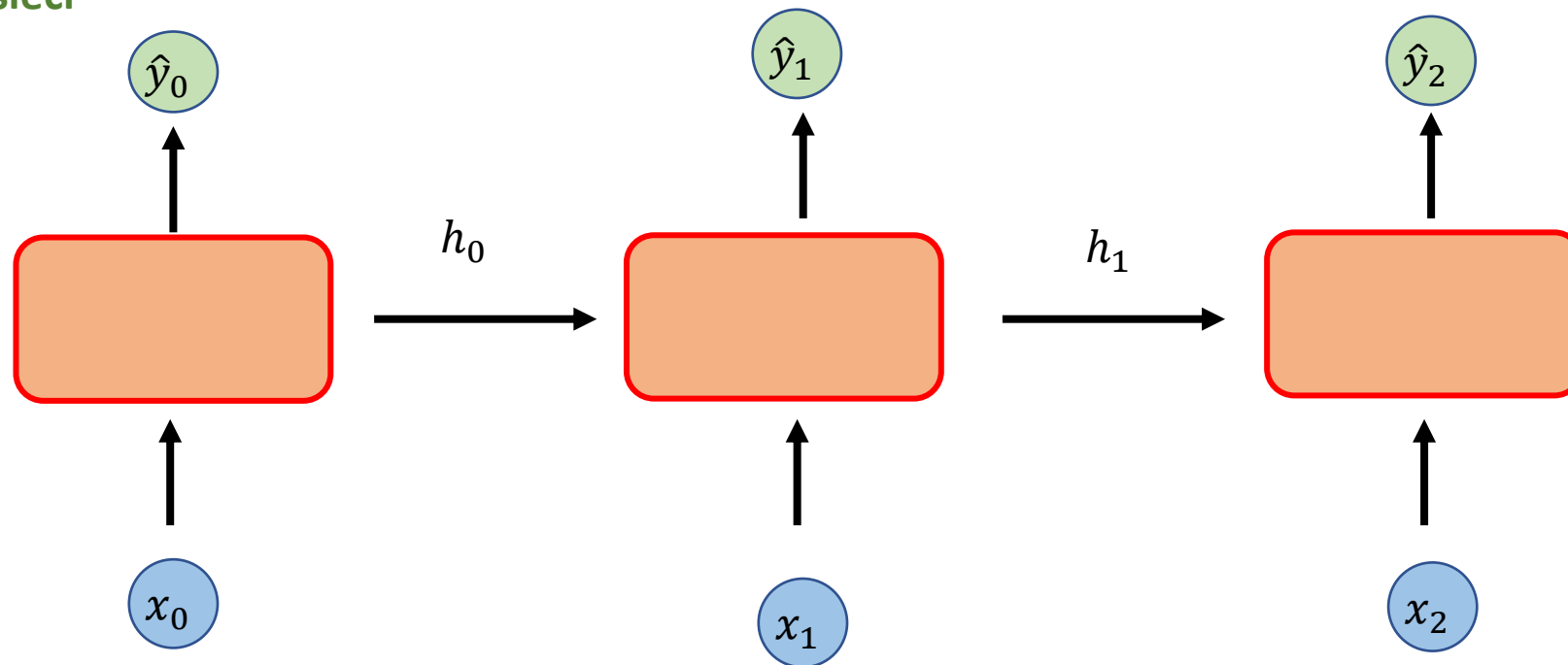
Wynik/output sieci



Wejście/input sieci

Modelowanie danych sekwencyjnych

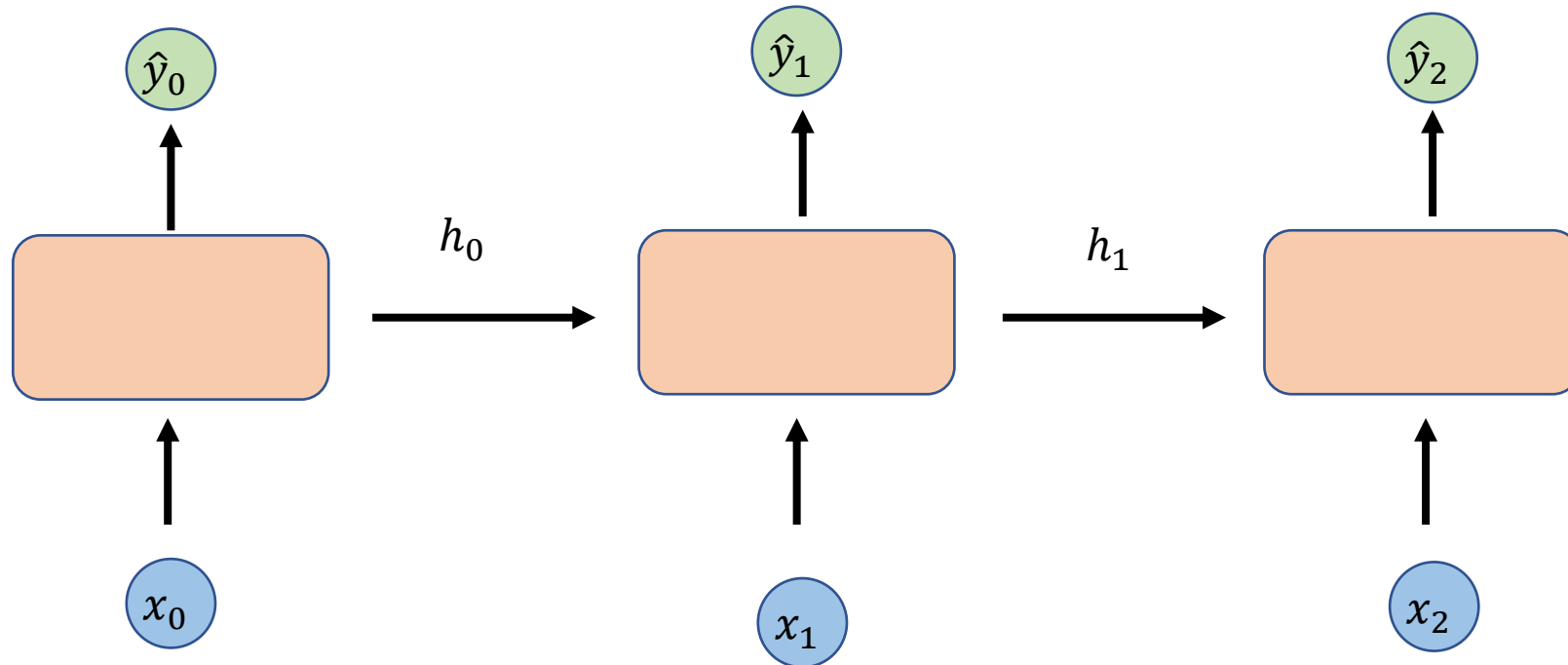
Wynik/output sieci



Wejście/input sieci

$$\hat{y}_t = f(x_t)$$

Architektura RNN



$$\hat{y}_t = f(x_t, h_{t-1})$$

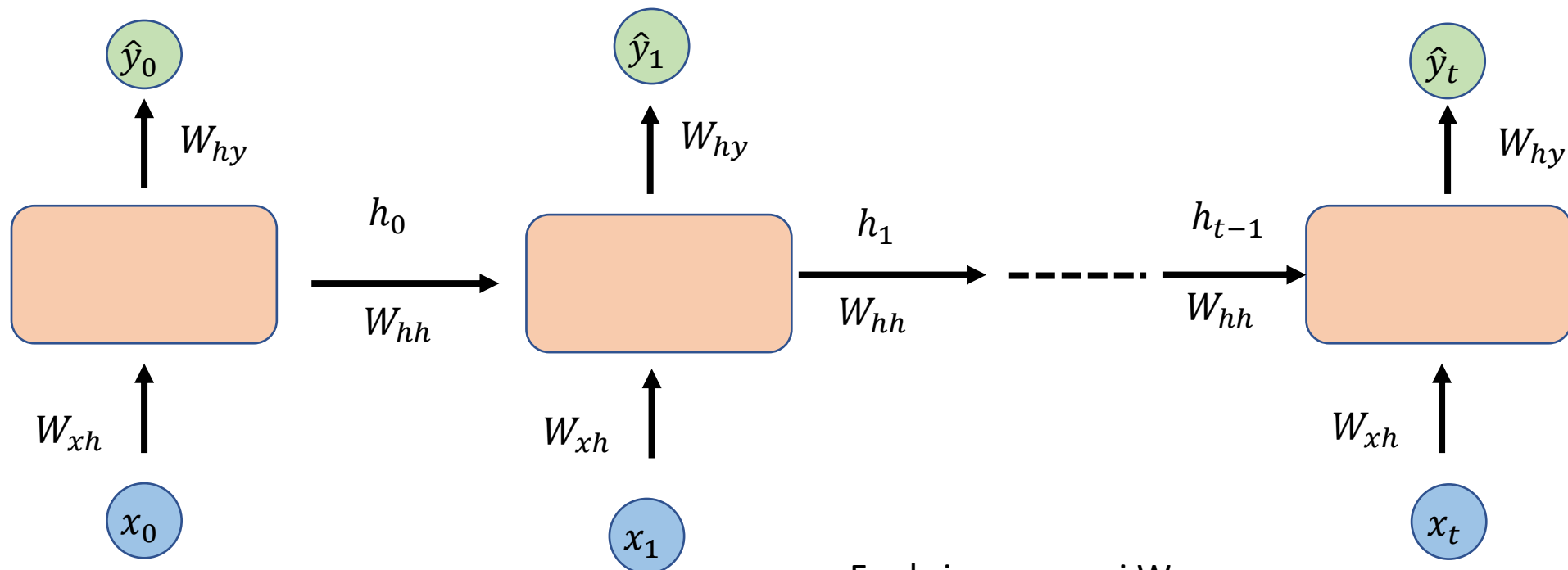
$$\boxed{\hat{y}_t} = f(\boxed{x_t}, \boxed{h_{t-1}})$$

Wynik/output sieci

Wejście/input sieci

Poprzednia pamięć

Te same wagi i funkcje aktywacji są dla każdego kroku czasowego w danej warstwie.



Funkcja z wagami W

$$h_t = f_W(x_t, h_{t-1})$$

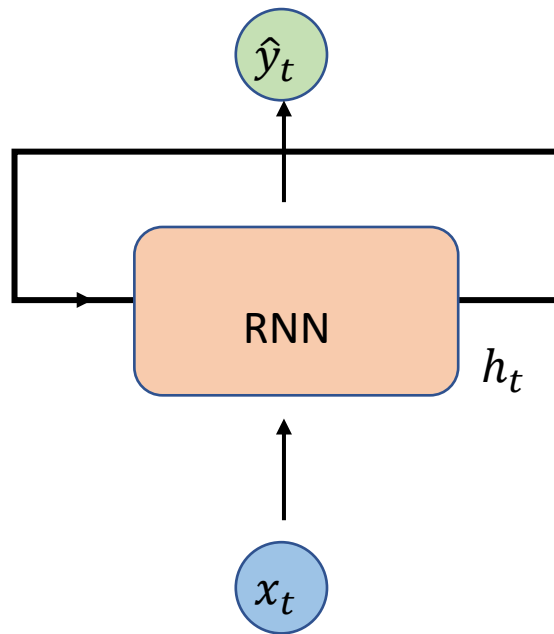
Komórka stanu/cell state

Wejście/input sieci

Poprzednia komórka stanu

Architektura RNN

Wynik sieci



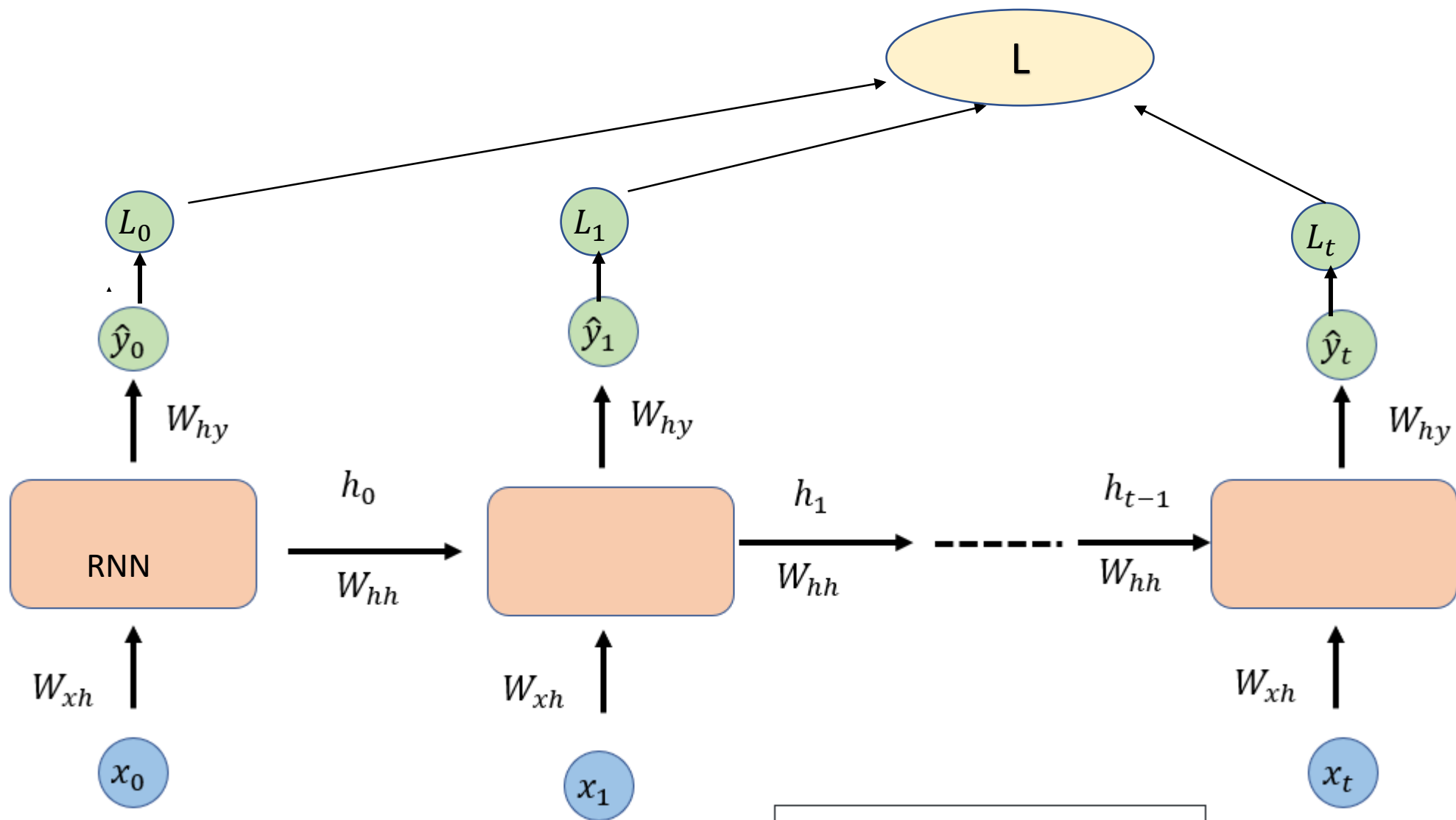
Wejście x_t

$$\hat{y}_t = W_{hy}^T h_t$$

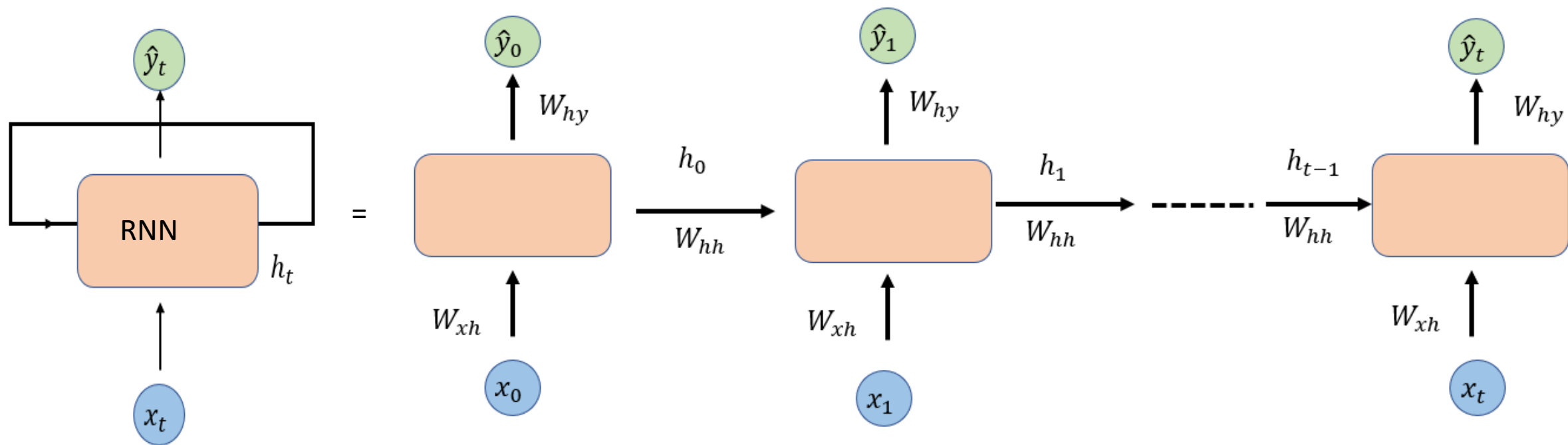
Wynik sieci

RNN mają komórkę stanu, która jest aktualizowana w każdym kroku czasowym.

$$h_t = \varphi(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$



$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}^{(T)}}{\partial W} \Big|_{(t)}$$



Wyjście pojedynczego neuronu

- ■ każdy neuron rekurencyjny ma dwa zestawy wag:
 - $\mathbf{w}_x$: dla wejść $\mathbf{x}^{(t)}$
 - $\mathbf{w}_y$: dla wyjść z poprzedniego kroku czasu $\mathbf{y}^{(t-1)}$
- ■ dane wyjściowe pojedynczego neuronu można obliczyć wg równania:

$$\mathbf{y}^{(t)} = \phi(\mathbf{x}^{(t)T} \cdot \mathbf{w}_x + \mathbf{y}^{(t-1)T} \cdot \mathbf{w}_y + b)$$

- gdzie $\phi(\cdot)$ jest funkcją aktywacji, np. ReLU, a b to *bias*

Zapis wektorowy

$$\begin{aligned}\mathbf{Y}^{(t)} &= \phi(\mathbf{X}^{(t)} \cdot \mathbf{W}_x + \mathbf{Y}^{(t-1)} \cdot \mathbf{W}_y + b) = \\ &= \phi([\mathbf{X}^{(t)} \cdot \mathbf{Y}^{(t-1)}] \cdot \mathbf{W} + b)\end{aligned}$$

gdzie

$$\mathbf{W} = \begin{bmatrix} \mathbf{W}_x \\ \mathbf{W}_y \end{bmatrix}$$

RNN

```
CLASS torch.nn.RNN(input_size, hidden_size, num_layers=1,  
                  nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0,  
                  bidirectional=False, device=None, dtype=None) \[SOURCE\]
```

Parameters

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a *stacked RNN*, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1
- **nonlinearity** – The non-linearity to use. Can be either `'tanh'` or `'relu'`. Default: `'tanh'`
- **bias** – If `False`, then the layer does not use bias weights b_{ih} and b_{hh} . Default: `True`
- **batch_first** – If `True`, then the input and output tensors are provided as $(batch, seq, feature)$ instead of $(seq, batch, feature)$. Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: `False`
- **dropout** – If non-zero, introduces a *Dropout* layer on the outputs of each RNN layer except the last layer, with dropout probability equal to `dropout`. Default: 0
- **bidirectional** – If `True`, becomes a bidirectional RNN. Default: `False`

Pseudokod dla RNN

```
state_t = 0                                ← The state at t  
for input_t in input_sequence:             ← Iterates over sequence elements  
    output_t = f(input_t, state_t)  
    state_t = output_t                     ← The previous output becomes the state for the next iteration.
```

```
state_t = 0  
for input_t in input_sequence:  
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)  
    state_t = output_t
```

```
from keras.layers import SimpleRNN
```

Przygotowanie danych

- Samples. Są to wiersze w danych. Jedna próbka może być jedną sekwencją.
- Time steps. Są to przeszłe obserwacje cechy, takie jak zmienne opóźnione.
- Features. Cechy. Są to kolumny w danych.

Sieci LSTM (Long Short-Term Memory)

Przykłady architektur LSTM

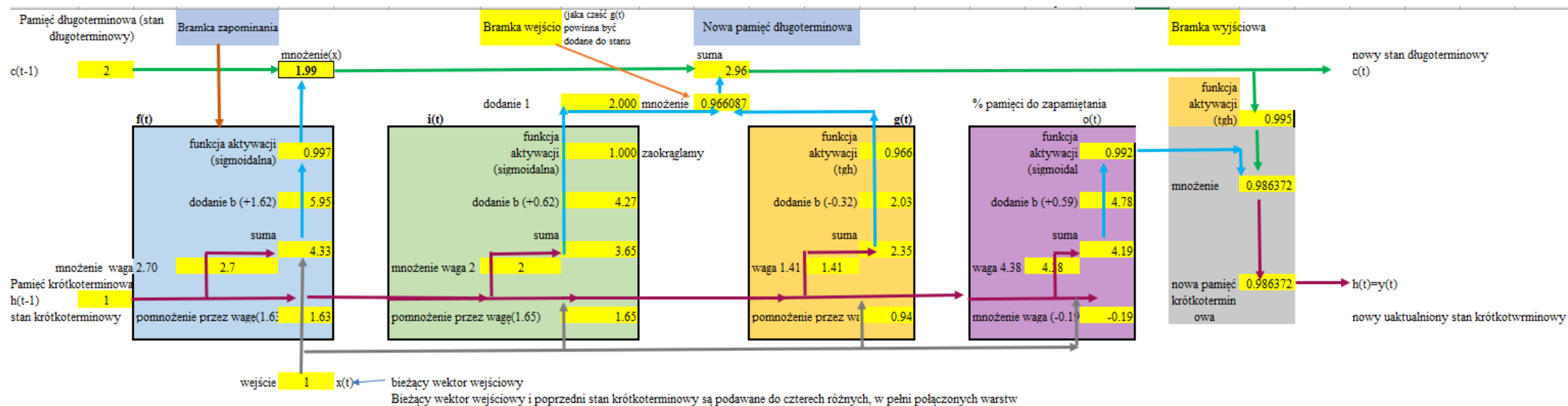
- Vanilla LSTM
- Stacked LSTM (stos)
- Bidirectional LSTM
- CNN-LSTM
- convLSTM
- Encoder-Decoder LSTM

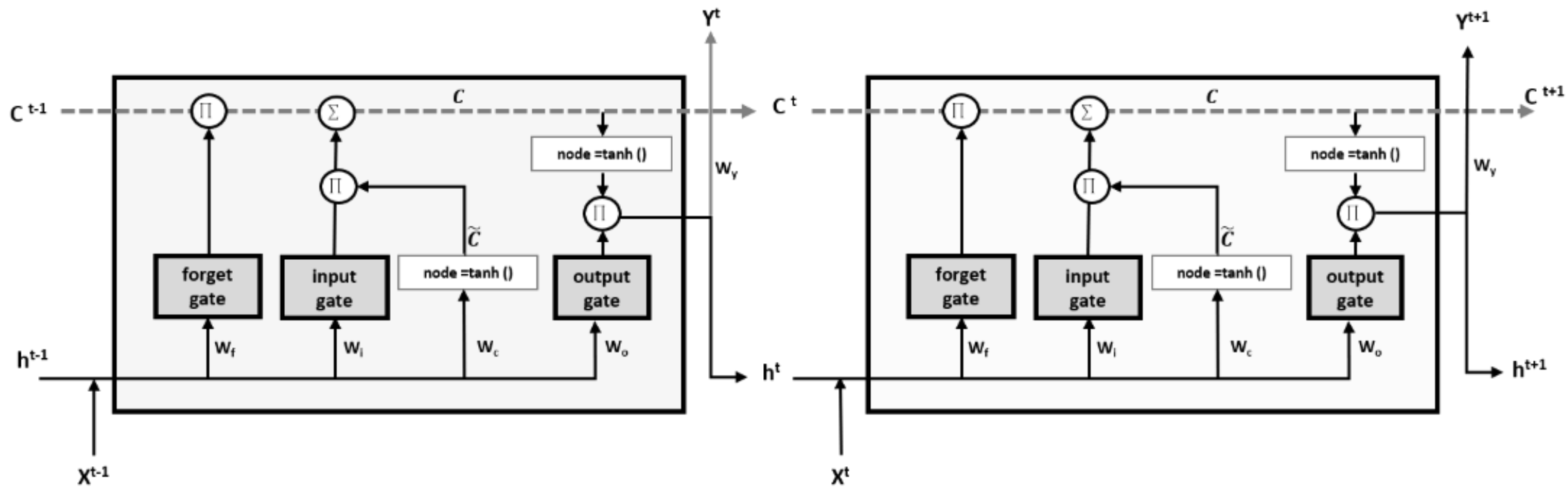
Trochę historii: Około 2006 roku dwukierunkowe LSTM zaczęły rewolucjonizować rozpoznawanie mowy, przewyższając tradycyjne modele w niektórych zastosowaniach. Ulepszyły one również rozpoznawanie mowy o dużym zakresie słownictwa i syntezę mowy na tekst i były używane w wyszukiwaniu głosowym Google i dyktowaniu na urządzeniach z Androidem.

Model LSTM-long short-term memory

- Model LSTM obejmuje specjalne komórki zdolne do uczenia się długoterminowych relacji. Architektura sieci LSTM opiera się na 3 typach bramek, które aktualizują i kontrolują stany komórek, są to bramka zapomnienia, bramka wejściowa i bramka wyjściowa.
- Bramki wykorzystują funkcje aktywacji sigmoidalnej i tanh.

Budowa komórki LSTM (memory unit)





LSTM

```
CLASS torch.nn.LSTM(input_size, hidden_size, num_layers=1, bias=True,  
                    batch_first=False, dropout=0.0, bidirectional=False, proj_size=0,  
                    device=None, dtype=None) \[SOURCE\]
```

Parameters

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two LSTMs together to form a *stacked LSTM*, with the second LSTM taking in outputs of the first LSTM and computing the final results. Default: 1
- **bias** – If `False`, then the layer does not use bias weights b_{ih} and b_{hh} . Default: `True`
- **batch_first** – If `True`, then the input and output tensors are provided as $(batch, seq, feature)$ instead of $(seq, batch, feature)$. Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: `False`

Przykładowy Model LSTM

```
class Model(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)  
        self.linear = nn.Linear(50, 1)  
    def forward(self, x):  
        x, _ = self.lstm(x)  
        x = self.linear(x)  
        return x
```

Liczba neuronów warstwy wyjścia

Liczba kroków czasowych np.
opóźnień

Liczba cech/zmiennych

Liczba cech

Liczba warstw

```
lookback = 4  
X_train, y_train = create_dataset(train, lookback=lookback)  
X_test, y_test = create_dataset(test, lookback=lookback)
```

	0	1	2	3
0	112.0	118.0	132.0	129.0
1	118.0	132.0	129.0	121.0
2	132.0	129.0	121.0	135.0
3	129.0	121.0	135.0	148.0
4	121.0	135.0	148.0	148.0
...	...	...	...	...
87	313.0	318.0	374.0	413.0
88	318.0	374.0	413.0	405.0
89	374.0	413.0	405.0	355.0
90	413.0	405.0	355.0	306.0
91	405.0	355.0	306.0	271.0

Jeśli dane mają postać (batch, sequence, features), należy utworzyć LSTM z argumentem `batch_first=True` (domyślnie jest to `False`).

W przeciwnym razie musisz przekazać dane do LSTM w postaci (sequence, batch, features), a otrzymasz dane wyjściowe w postaci (sequence, batch, features_out).

Trenowanie modelu

Sieć jest trenowana przy użyciu algorytmu wstecznej propagacji w czasie (**Backpropagation Through Time**)

i optymalizowana zgodnie z algorytmem optymalizacji
i funkcji straty określonej podczas kompilacji modelu.

<https://www.geeksforgeeks.org/ml-back-propagation-through-time/>

Przykłady zastosowania sieci LSTM

Szeregi czasowe

- **Prognozowanie jednowymiarowych szeregów czasowych** (many-to-one model);

Jedna seria danych (szereg czasowy) i chcemy przewidzieć jeden krok czasowy poza sekwencją wejściową. Model wiele-do-jednego.

- **Wielowymiarowe prognozowanie szeregów czasowych;**

Mamy wiele serii z wieloma wejściowymi krokami czasowymi i chcemy przewidzieć jeden krok czasowy poza jedną lub więcej sekwencji wejściowych. Model wiele-do-jednego. Każda seria jest po prostu kolejną cechą/zmienną wejściową.

- **Klasyfikacja szeregów czasowych np.** zapis EKG pewnego pacjenta-ma chorobę serca czy nie.

Przykłady zastosowania sieci LSTM

Automatyczne podpisywanie obrazów - zadanie, w którym, biorąc pod uwagę obraz, system musi wygenerować opis, który opisuje zawartość obrazu.

Automatyczne tłumaczenie tekstu.

Generowanie pisma odręcznego.

Generowanie odpowiedzi do pytań zadanych jako tekst.

Klasyfikacja/opis ruchu-generowanie opisu do ciągu gestów.

Klasyfikacja obrazów.

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>